Open Sheet

Operasi Bit Vektor

Operasi	Tanda
And	&
Or	|
Not	~
XOR	^

XOR:

^	0	1
0	0	1
1	1	0

Operasi Logic

Operasi	Tanda
And	&&
Or	||
Not	!

Result :
True : 0x01
False : 0x00

Operasi Shift

Operasi	Tanda	Ket.
Left shift	<<	$\times 2^k$
Right shift	>>	$\div 2^k$

Fill missing value with 0 (zero) for logic operation, 1 for arith.

- Short in C is 2 byte long
- Mappings between unsigned and two's complement number : keep bit representations and reinterpret.
- By default constants are considered to be signed integer.
- If there is a mix of unsigned and signed in single expression, signed values implicitly cast to unsigned.

Sign Extension

- Make k copies of sign bit.

Expanding (short to int)
↳ U : add 0
↳ S : add sign extension

Truncating (int to short)
↳ Reinterpret.

Floating point Representation

Numerical form :
$(-1)^s \cdot M \cdot 2^E$
S : Sign bit
M : Mantissa
E : Exponential

$E = Exp - bias$

Single precision (32-bit)
S : 1 bit
M : 8 bit
E : 23 bit

Double precision (64-bit)
S : 1 bit
M : 11 bit
E : 52 bit

"Normalized value"
↳ exp ≠ 000 and exp ≠ 111..

"Denormalized value"
↳ exp = 00..0
↳ E = −bias + 1

Sign	Exp	Frac

"Infinity"
↳ Exp = 11..1
↳ Frac = 00..0

"NaN"
↳ Exp = 11..1
↳ Frac ≠ 00..0

FP Multiplication

↳ S : $s_1 \wedge s_2$
↳ M : $M_1 \times M_2$
↳ E : $E_1 + E_2$
↳ Fixing :
If M ≥ 2, shift M right, increment E
If M < 1, M left to k position, decrement E by k
Overflow if E out of range
Round M to fit frac position.

Integer Register (IA32)

%eax	%ax	
%ecx	%cx	
%edx	%dx	
%ebx	%bx	
%esi	%si	
%edi	%di	
%esp	%sp	
%ebp	%bp	

16-bit
2-byte
32-bit
4-byte

Operan :
- Imm (\$0x...)
- Register (%...)
- Memory ((%...))

Memory ↛ Memory

push_ : Push
mov_ : Move
pop_ : Pop
Ret : Return
lea_ : Load effective address

D(Rb, Ri, S)
Mem[Reg[Rb] + S × Reg[Ri] + D]
Cases :
(Rb, Ri), D(Rb, Ri), (Rb, Ri, S)
D : Constant
Rb : Base register
Ri : Index register
S : Scale

Sal : Shift arithmetic left
Shl : Shift logical left

Sar : Shift arithmetic right
Shr : Shift logical right

Sar count, dst
Sal count, dst

movsbl => Signed
movzbl => Unsigned
imul => Multiplication (Signed)
mulq => Unsigned multiplication
neg dst => negative
idivq => Signed divide
j => Jump
Jmp => Jump to label
jne => Jump not equal
js => Jump negative
jmp => Jump
pop => Remove from stack.
not => Not

2^n	Nilai
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024

Hexadecimal

Desimal	Hexa
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

2^{-n}	$1/2^n$	Nilai
1	1/2	0,5
2	1/4	0,25
3	1/8	0,125
4	1/16	0,0625
5	1/32	0,03125
6	1/64	0,015625
7	1/128	0,0078125

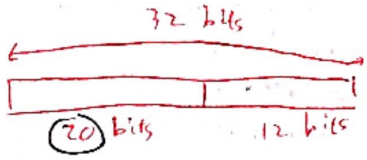
P.A. bits = $\log_2 2^n$ B, dimana n = ukuran memori

Block offset = $\log_2 2^n$ B, dimana n = block size

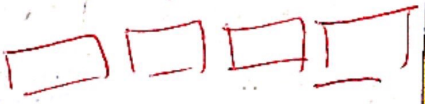
Block number = $\log_2 (2^n/2^m)$ B, dimana n = P.A. dan m block offset size

Line number = $\log_2 (2^n/2^m)$ B, dimana n = Block number, dan m = block offset size

ex 1: RAM size = 4 GB
cache size = 1 MB
Block size = 4 KB



P.A. = $2^2 \times 2^{30}$ B = 2^{32} B
= $\log_2 2^{32}$ B = 32 Bits



Block offset = $\log_2 (2^2 \times 2^{10})$ B = $\log_2 2^{12}$ B = 12 bit
Block number = $\log_2 (2^{32}/2^{12})$ B = $\log_2 2^{20}$ B = 20 bit
Line number = $\log_2 (2^{20}/2^{12})$ B = $\log_2 2^8$ B = 8 bit

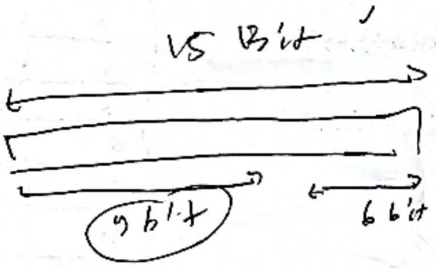
1
2

$2^9 / 2^6 \times 2^3$ $2^9 / 2^3$ $1 = 0$

latency
miss rate

cache

$\log_2 2^0 = 1$

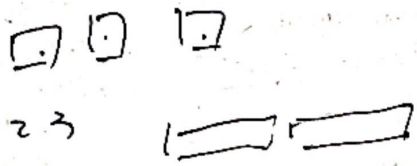


(7) +6
(7+7) +6
+6

Ukuran cache 32 KB
8 way
Block size 64 byte
Alamat memori 32 bit

$\log_2 (2^9/2^3 \times 2^6)$
$\log_2 (2^9/2^9)$
$\log_2 (2^0) = 1$ bit

$2^{15} = 2^k \times 2^6$
$2^9 \times$



a[0][0] a[1][0]
a[0][1] a[1][1]
a[0][2] :
a[0][3]

	0	1	2	3
0	m	m	m	m
1				
2				

$2^9 = 2^{9}$

C = ~~sx~~ xD